STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: JAVA_Q012_ROLE
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: OOPS Concepts using Java with AI	STATUS: In Progress

Practice Worksheet – Q012 – Role-Based Access Details Generator

Engineer a role-based access control (RBAC) hierarchy modeling Admin, Manager, and Employee users with permission bitmasks and role descriptions.

KEY LEARNING OBJECTIVES:

- Model role hierarchies and permission inheritance in Java.
- Apply SOLID object-oriented design principles and access control.
- Implement defensive parameter validation and error-free edge handling.
- Verify correctness with standard JUnit test assertions and execution logs.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Review the business domain requirements and model the class structure.
Step 2: Implement private attributes, constructor overloading, and accessors.
Step 3: Implement the core computation and validation methods in clean Java.
Step 4: Document test execution examples and verify edge-case behavior.

EXERCISE 12.1: DOMAIN MODELING & CLASS ARCHITECTURE (USER ROLE CLASS HIERARCHY)

Problem Statement: Define the core class architecture for Q012 – Role-Based Access Details Generator. Establish private attributes, access modifiers, constructors, and encapsulated getter/setter methods with precondition checks.

Reference Baseline / Scenario / Code:

```
public class Q012_Generate_Role_Based_Details { private double amount; ... }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Class Attributes & Modifiers: _____

[2] Constructor Implementation: _____

[3] Encapsulation & Preconditions: _____

EXERCISE 12.2: CORE BUSINESS LOGIC & COMPUTATION (PERMISSION BITMASK OPERATIONS)

Problem Statement: Implement the core algorithmic method for Q012 – Role-Based Access Details Generator. Formulate the calculation logic, boundary edge validations, and state updates.

Reference Baseline / Scenario / Code:

```
public double computeResult() { // Algorithm logic for Q012 – Role-Based Access Details Generator return calculatedValue; }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Algorithm Steps & Formula: _____

[2] Conditional Branching Rules: _____

[3] Method Implementation Code: _____

EXERCISE 12.3: EDGE-CASE HANDLING & BOUNDARY SAFEGUARDS (ACCESS CONTROL VERIFICATION)

Problem Statement: Identify 3 critical edge cases for Q012 – Role-Based Access Details Generator (e.g. zero values, negative numbers, extreme thresholds). Implement defensive guard clauses and error handling.

Reference Baseline / Scenario / Code:

```
if (value <= 0) throw new IllegalArgumentException("Invalid input value");
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Edge-Case Boundary Matrix: _____

[2] Defensive Guard Clauses: _____

[3] Validation Logic Implementation: _____

EXERCISE 12.4: TESTING & VERIFICATION EXECUTION (PROFILE DETAIL FORMATTING)

Problem Statement: Construct a JUnit 5 test method verifying that Q012 – Role-Based Access Details Generator computes expected results accurately across normal, boundary, and invalid scenarios.

Reference Baseline / Scenario / Code:

```
@Test void testCalculation() { assertEquals(expected, service.compute(input)); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] JUnit 5 Test Annotations: _____

[2] Assertion Statements: _____

[3] Verification Output Log: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Encapsulation & Access:** All fields private; access mediated by validated methods.
- [ ] **Algorithmic Correctness:** Business logic computes exact expected outputs across all slabs.
- [ ] **Edge Safeguards:** Defensively guards against zero, negative, and extreme inputs.
- [ ] **Code Standards:** Adheres to standard Java naming conventions and clean code practices.
- [ ] **Test Verification:** JUnit test assertions verify deterministic execution.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *How does encapsulation protect the internal state of Q012 – Role-Based Access Details Generator from unauthorized external corruption?*
- *Why is it essential to test boundary values at the exact tariff or discount transitions?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases